DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BiLSTM NETWORK

A PROJECT REPORT

Submitted by

1.	**ARAVINDHAN A**	**22AI002**
2.	**MADHAN V**	**22AI025**
3.	**PREM KUMAR R**	**22AI039**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

NANDHA ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

ERODE – 638 052

MAY 2026

NANDHA ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

ERODE - 638 052

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BiLSTM NETWORK"** is the bonafide record of project work done by **ARAVINDHAN A (22AI002), MADHAN V (22AI025), PREM KUMAR R (22AI039)** for the award of the Bachelor of Technology in **Artificial Intelligence and Data Science** of Anna University, Chennai during the year 2025 - 2026.

SIGNATURE OF THE SUPERVISOR
MS. M. PARVATHI, M.E., Ph. D.,
ASSISTANT PROFESSOR,
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE.
NANDHA ENGINEERING COLLEGE,
ERODE 638 052.

SIGNATURE OF THE HOD
DR. P. KARUNAKARAN, M.E., PH.D.,
PROFESSOR & HEAD,
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE.
NANDHA ENGINEERING COLLEGE,
ERODE 638 052.

Submitted for the End Semester 22AID01- Project Work Viva-Voce Examination held on ……

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

First and foremost, we express our heartfelt gratitude to our parents for giving us the opportunity to pursue and successfully complete this engineering course.

We wish to express profound gratitude to **Thiru. V. Shanmugan, Chairman,** Sri Nandha Educational Trust, **Thiru. S. Nandhakumar Pradeep, Joint Managing Trustee,** Sri Nandha Educational Trust and **Thiru. S. Thirumoorthi, Joint Managing Trustee,** Sri Nandha Educational Trust for providing opportunities in all possible ways for our improvement.

We wish to convey our gratefulness to our beloved **Principal, Dr. U. S. Ragupathy, Ph.D.** for his strong support and motivation towards a great level of success in our career.

We take this opportunity to express our thanks to our beloved **Head of the Department Dr. P. Karunakaran, M.E., Ph.D., Professor, Department of Artificial Intelligence and Data Science**, for his help and encouragement.

We articulate our genuine thanks to our **Project Coordinator, Dr. K. Lalitha, M.Tech., Ph.D., Professor, Department of Artificial Intelligence and Data Science**, who have been the key spring of motivation to us throughout the completion of our course and project work.

We sincerely thank our **Supervisor Ms. M. Parvathi, M.E., Ph.D., Assistant Professor, Department of Artificial Intelligence and Data Science**, for her valuable guidance to complete this project.

We express our sincere thanks to all **Artificial Intelligence and Data Science** Department Faculty Members for their help in completing this project.

LIST OF CHAPTERS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

ABBREVIATIONS	EXPANSION
AI	Artificial Intelligence
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
FN	False Negative
FP	False Positive
GDFA	Generalized Deepfake Audio
LSTM	Long Short-Term Memory
MFCC	Mel-Frequency Cepstral Coefficients
ML	Machine Learning
ROC-AUC	Receiver Operating Characteristic – Area Under Curve
SER	Speech Embedded Representations
TN	True Negative
TP	True Positive
TSS	Two-Stem Splitter
TTS	Text-to-Speech
VC	Voice Conversion
XAI	Explainable Artificial Intelligence
DL	Deep Learning
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
CPU	Central Processing Unit
GPU	Graphics Processing Unit
API	Application Programming Interface
STFT	Short-Time Fourier Transform

ABSTRACT

The recent spike in the advancement of speech synthesis and voice conversion technologies has created a torrent of realistic deepfake audio outputs which pose serious threats to personal privacy, media integrity, and cybersecurity. This project presents a Generalized Deepfake Audio (GDFA) detection framework that incorporates self-supervised speech-based embedded representations (SER) combined with BiLSTM (Bidirectional Long Short-Term Memory) and LSTM classifiers alongside a two-stem splitter (TSS) solution. Pre-trained self-supervised speech models extract high-level speech embeddings, temporal speech models are trained using BiLSTM and LSTM networks, and the TSS produces more accurate speech print separation between genuine and artificial voice prints compared to spectrogram-based methods.

The GDFA framework was trained and evaluated on the large-scale Fake-or-Real dataset, which contains both TTS-generated and human-recorded speech samples from diverse speakers and recording conditions. Results demonstrate that the BiLSTM-based classifier achieves 95% accuracy, outperforming conventional spectrogram-based CNN approaches (88%). The proposed framework exhibits low computational complexity relative to traditional methods, cross-dataset generalization, and is deployed via an interactive Streamlit web application enabling real-time deepfake audio detection suitable for digital forensics, media verification, and cybersecurity applications.

The proposed system contributes to the Sustainable Development Goals (SDGs), particularly aligning with **SDG 9 – Industry, Innovation and Infrastructure** by developing an AI-based solution for detecting deepfake audio and strengthening digital security. It also supports **SDG 16 – Peace, Justice and Strong Institutions** by preventing the misuse of synthetic audio in fraud, impersonation, and misinformation, thereby improving trust and security in digital communication systems.

CHAPTER 1

INTRODUCTION

The fast progress of speech synthesis and voice conversion technologies has resulted in artificial speech that seems disturbingly similar to human speech — what is more easily recognized as audio deepfakes. In addition to posing a serious risk to privacy, social trust, and cybersecurity, bad actors are growing their reliance on synthetic voices to commit impersonation fraud, misinformation campaigns, identity theft, automated scam calls, and circumventing biometric authentication methods.

Modern Text-to-Speech (TTS) and Voice Conversion (VC) systems generate synthetic speech with the help of deep learning and demonstrate the same prosodic, tonal, and emotional properties as human speech. These systems are incredibly hard to detect by individuals lacking specialized expertise, making automated detection systems an urgent necessity.

Due to the growing naturalism and realism of synthetic speech, scientists must devise reliable and computationally efficient solutions for detecting deepfakes in digital forensics and secure communication settings. Research has explored various methods including CNN-based models relying on spectrogram data, manual feature extraction, self-supervised learning, and ensemble classification. However, most modern solutions are operationally restricted by complexity or designed to work with small sets of data with no capacity to extrapolate to different generation methods.

The proposed work designs a lightweight and general method that detects synthetic audio by combining high-level speech embedding models with BiLSTM/LSTM-based classifiers. The vision is to maximize detection accuracy, generalization, and the ability to counter unseen deepfake attacks while ensuring computational efficiency. The system incorporates self-supervised learning with temporal sequence modeling to provide a stable framework for detecting AI-produced speech in the real world.

1.1 OUTLINE OF PROJECT

The report is structured into several chapters to present a clear and systematic explanation of the deepfake audio detection system. The initial chapter introduces the concept, highlights the problem, and states the objectives of the work. The literature survey section reviews existing methods and recent advancements in deepfake detection techniques.

Subsequent chapters describe the system development process, including methodology, workflow, and overall design. Details about the technologies used, system modules, and implementation approach are also provided. The later sections focus on results, performance evaluation, and practical applications of the system. The report concludes with key findings and future enhancements, ensuring a complete understanding of the system and its functionality.

1.2 OBJECTIVE

The primary objective is to develop an intelligent and reliable system capable of detecting deepfake audio with high accuracy and efficiency. The system focuses on analyzing speech signals using advanced deep learning techniques, including self-supervised speech embeddings combined with LSTM and BiLSTM models. These techniques help in capturing both low-level acoustic features and high-level temporal patterns present in audio data.

Another important objective is to enhance the system's ability to distinguish subtle differences between genuine and artificially generated speech. By improving feature representation and sequence modeling, the system aims to reduce false positives and false negatives, ensuring more dependable predictions. The design also emphasizes computational efficiency so that the system can perform well in real-time environments.

In addition, the system aims to provide a simple and user-friendly interface that allows users to upload or record audio easily and obtain quick results. The overall goal is to create a practical solution that can be effectively used in domains such as cybersecurity, media authentication, and digital forensics, where verifying audio authenticity is crucial.

1.3 SCOPE

The scope includes the design and implementation of a deep learning-based framework that can classify audio samples as real or synthetic. The system is capable of processing various types of audio inputs and analyzing them using advanced feature extraction and sequence modeling techniques. It is intended to provide reliable predictions even when dealing with diverse datasets and different speech conditions.

The system can be applied in multiple real-world scenarios, such as detecting fraud in voice-based communication, verifying the authenticity of media content, and enhancing the security of voice recognition systems. It can also support digital forensic investigations by helping analysts identify manipulated audio evidence.

Furthermore, the framework is designed to be scalable and adaptable, allowing integration with future technologies and improved models. However, the scope also recognizes certain limitations, such as sensitivity to background noise, compression effects, and the need for continuous updates to handle newly emerging deepfake generation techniques. Despite these challenges, the system provides a strong foundation for developing advanced audio verification solutions.

CHAPTER 2

LITERATURE SURVEY

2.1 REVIEW OF RELATED WORKS

Late discoveries in the detection of audio deepfakes have indicated that artificial speech has quite a number of differences with genuine speech in factors such as prosody, fine motor articulation, and subtle alterations in timing. These variations form a strong foundation in designing successful detection models that can distinguish natural human vocal from artificial voices. Many researchers remark on the need to have explainable detection methods especially in forensics and key security contexts, where comprehending model decisions is as important as detection accuracy.

Audio deepfake detectors have been evaluated on numerous benchmark datasets and across various TTS systems to evaluate spectral features, CNN-based approaches, and combinations of deep learning algorithms. Evaluations have shown persistent vulnerabilities to replay attacks, reduced performance under background noise, and lower stability with multi-channel or recorded audio. Consequently, scholars emphasize the importance of small and computationally efficient detectors capable of running in real-time and protecting user privacy.

Verma et al. (2025) provided a comparative study demonstrating that deepfake audio is progressively becoming a serious concern as deep learning models develop, though current models continue to face generalization issues across different datasets. Ahmad et al. (2025) applied deepfake audio detection to the Urdu language, highlighting the need for language-specific models and datasets to combat speech forgeries in low-resource conditions. Hamza et al. (2022) demonstrated that MFCC-based framework approaches remain useful in reflecting differences between original and synthetic voices.

Rehman et al. (2023) introduced the ELAD-SVDSR dataset to enable real-time assessments for deepfake detection and speaker identification using extended audio recordings. Bohara and Bairwa (2025) showed that spectrogram-based learning strategies demonstrate the importance of time-frequency representation for conceptualizing artifacts introduced during deepfake synthesis. Lee et al. (2025) improved detection capability through dual-channel approaches capturing both direct and reflected sound, facilitating capture of environmental input not captured by other systems.

Patel et al. (2023) provided a case study on deepfake generation and detection, indicating that the arms race between generation and detection technologies is perennial in nature. Zaman et al. (2024) demonstrated that hybrid transformer architectures incorporating diverse spectral and temporal audio features enhance deepfake speech detection performance. Kim et al. (2025) tackled detection system robustness by adding noise during inference to improve resistance to adversarial manipulation and previously unseen audio sources.

Can and Soyhan (2025) presented data regarding spectro-temporal CNN fusion models and suggested methods to monitor deepfake speech and attribute manipulation to source systems. El Kheir et al. (2025) proposed combining spectral representations with self-supervised representations as a feature fusion strategy to enhance detection resilience against unseen deepfake attacks. Hao et al. (2025) proposed a two-stage learning approach combining hierarchical experts for effective pre-training and strong generalization in deepfake audio detection systems.

Rimon et al. (2025) showed that robustness of detection mechanisms improves when data and feature variability are enhanced across various deepfake generation techniques. Dhivya et al. (2025) demonstrated that hybrid CNN-LSTM architectures provide substantial performance enhancement compared to using individual deep learning components, influencing the utilization of hybrid networks in detection-based tasks

Author (Year)	**Method**	**Key Contribution**	**Limitation**
Verma et al. (2025)	Deep Learning Models	Comparative study across advanced DL models for audio deepfake detection	Generalization issues across datasets
Ahmad et al. (2025)	Deep Neural Networks	Deepfake detection for Urdu low-resource language	Language-specific; limited cross-lingual transfer
Hamza et al. (2022)	MFCC + ML	Traditional acoustic features for deepfake classification	Limited to seen TTS systems

Zaman et al. (2024)	Hybrid Transformer	Spectral and temporal feature fusion for deepfake speech classification	High computational cost
Kim et al. (2025)	Noise Injection	Adversarial robustness for audio deepfake detection	Performance drop on clean audio
El Kheir et al. (2025)	Feature Fusion	Self-supervised + spectral fusion for robust detection	Requires pre-trained model infrastructure
Hao et al. (2025)	Two-stage Learning	Hierarchical expert fusion for generalized deepfake detection	Complex training pipeline

Table 2.1: Summary Of Related Research Works

2.2 EXISTING SYSTEMS

Deepfake audio detection has evolved significantly with the advancement of speech synthesis technologies. Earlier methods mainly relied on traditional signal processing techniques such as MFCC features and spectrogram analysis. These methods used classical machine learning algorithms to classify audio as real or fake. Although they provided basic detection capability, they were limited in capturing complex speech patterns and often failed against highly realistic synthetic audio.

With the emergence of deep learning, more advanced approaches such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) models were introduced. These models can automatically learn spatial and temporal features from audio data, improving detection accuracy. BiLSTM models further enhance performance by analyzing both forward and backward speech sequences.

Recent studies focus on self-supervised learning and transformer-based models, which extract high-level representations from audio signals. Hybrid approaches combining CNN, LSTM, and attention mechanisms have also been

proposed to improve robustness and generalization. Despite these improvements, challenges such as high computational cost, poor generalization, and sensitivity to noise still exist.

2.3 COMPARISON OF METHODS

The comparison between conventional and deep learning-based approaches highlights the advantages of modern techniques in detecting deepfake audio.

Parameter	**Traditional Methods**	**Deep Learning Methods**	**Proposed System**
Feature Extraction	Handcrafted (MFCC, Spectrogram)	Automatic feature learning	Self-supervised embeddings
Accuracy	Moderate	High	Very High
Temporal Learning	Limited	Good	Excellent (BiLSTM)
Generalization	Poor	Moderate	High
False Detection Rate	High	Moderate	Low
Real-Time Capability	Limited	Moderate	High
Scalability	Low	Moderate	High

Table 2.2: Comparison of Deepfake Audio Detection Methods

2.4 RESEARCH GAPS AND PROPOSED SOLUTION

Research Gaps

The analysis of existing methods reveals several key limitations:

- Difficulty in detecting deepfake audio in real-time
- Limited ability to capture temporal dependencies in speech
- Poor generalization to new and unseen deepfake techniques
- High computational complexity in advanced models
- Lack of user-friendly deployment systems

Proposed Solution

To address these challenges, an advanced deep learning-based approach is adopted using self-supervised speech embeddings combined with LSTM and BiLSTM models. This approach improves feature extraction and enhances temporal pattern analysis.

The system is designed to support real-time detection while maintaining high accuracy. It also provides an interactive interface for easy usage. By combining efficient feature representation with sequence modeling, the system reduces false predictions and improves reliability.

Gap vs Solution Table

Research Gap	**Proposed Solution**
No real-time detection	Real-time audio analysis system
Poor temporal feature learning	LSTM/BiLSTM sequence modeling
Low generalization	Self-supervised embeddings
High false predictions	Improved feature representation
Complex systems	Simplified and efficient architecture

Table 2.3: Research Gaps and Proposed Solutions

2.5 SUMMARY OF LITERATURE REVIEW

The literature review shows a clear transition from traditional signal processing techniques to advanced deep learning-based approaches for deepfake audio detection. While conventional methods provide basic detection, they are not sufficient for handling modern synthetic audio.

Deep learning models significantly improve detection accuracy by learning complex speech patterns and temporal relationships. However, challenges such as generalization, computational cost, and real-time performance still remain. The proposed system addresses these issues by integrating self-supervised feature extraction with LSTM/BiLSTM models, resulting in a more accurate, efficient, and scalable solution.

CHAPTER 3

PROBLEM DEFINITION

The development of intelligence and deep learning has changed how audio content is created and processed. Recently speech synthesis and voice conversion techniques have made realistic audio, known as deepfake audio. These synthetic audio samples mimic speech in tone, pitch, emotion and speaking style making them hard to distinguish from real recordings.

This technology has benefits like assistants, accessibility tools and entertainment. However it also raises security concerns. Deepfake audio can be misused for impersonation attacks, financial fraud spreading misinformation and bypassing voice-based authentication systems. The increasing accessibility of tools has made it easier for malicious users to generate fake audio.

One major challenge with deepfake audio is detection. Traditional methods rely on inspection or basic signal processing techniques. However these approaches are no longer effective due to the quality of modern deep learning models. There is a growing need for automated systems that can accurately distinguish between synthetic audio.

Another issue is the lack of generalization in existing detection systems. Many models are trained on limited datasets. Fail to perform well when exposed to new types of deepfake audio. This makes it difficult to deploy systems in real-world scenarios where fake audio is constantly evolving.

In addition to accuracy computational efficiency is a concern. Some existing solutions require processing power and are not suitable for real-time applications. This limits their usability in environments like live monitoring systems and online platforms.

To address these challenges this project focuses on developing a deepfake audio detection system. The system aims to capture both temporal characteristics of audio signals. This approach helps in identifying differences between real and fake speech.

Overall the problem addressed in this project is to design a efficient and scalable system that can detect deepfake audio with high accuracy. Such a system is essential for improving security and maintaining trust in communication.

3.1 EXISTING SYSTEM

Existing systems for deepfake detection rely on conventional machine learning and deep learning techniques. These systems extract low-level features from signals to classify audio as real or fake.

Spectrogram-based approaches convert signals into visual representations, which are then analyzed using CNN models. However these models often focus on surface-level features and fail to capture deeper temporal dependencies.

Another limitation of existing systems is their dependency on handcrafted features. Although these features are effective for audio classification tasks they are not sufficient to detect advanced deepfake audio.

Many existing models suffer from overfitting meaning they perform well on the training dataset but fail when tested on data. Computational complexity is also a drawback. Some systems require processing power and memory making them unsuitable for real-time applications.

Existing systems are also sensitive to noise and variations in recording conditions. Factors like background noise and recording devices can significantly affect the performance of these models.

Due to these limitations existing systems are not fully reliable for detecting deepfake audio in applications.

3.2 PROBLEM STATEMENT

The increasing sophistication of deepfake audio technologies has made it extremely difficult to differentiate between synthetic speech using traditional methods. Existing detection systems are limited in terms of accuracy, generalization and computational efficiency.

There is a need to develop a system that can effectively analyze audio signals and detect deepfake content with high precision. The system should be capable of identifying patterns and inconsistencies in speech that indicate synthetic generation.

The system must be able to handle audio inputs, including variations in language speaker characteristics and recording conditions. It should also be robust against noise and capable of detecting types of deepfake audio.

Another important requirement is real-time processing capability. The system should provide reliable predictions to support applications like cybersecurity monitoring and media verification.

To address these challenges the proposed system aims to utilize techniques. These methods enable the model to learn speech representations and temporal patterns improving detection accuracy and generalization.

In summary the problem is to design a deepfake audio detection system that's accurate, efficient and scalable. The system should be capable of handling real-world challenges and ensuring performance, across different scenarios.

CHAPTER 4

SYSTEM REQUIREMENTS

To make the Deepfake Audio Detection System work properly you need to have the hardware and software. This system needs to be able to handle audio use learning models and work with the web. So it is important to have a balance between hardware and software.

4.1. HARDWARE REQUIREMENTS

You need to know what kind of hardware is required to run the Deepfake Audio Detection System. The system can work with hardware but it will work better with more powerful hardware.

Component	**Minimum Requirement**	**Recommended**
Processor	Intel Core i5 / AMD equivalent	Intel Core i7 / AMD Ryzen 7 (8+ cores)
RAM	8 GB	16 GB or higher
Storage	20 GB SSD free space	50+ GB SSD (to accommodate large audio datasets and pre-trained weights)
GPU	Optional (CPU-only inference supported)	CUDA-capable GPU (e.g., NVIDIA RTX series) for faster embedding extraction and model training
Microphone	Standard built-in microphone	Noise-controlled external microphone for high-quality live audio capture

Table 4.1: Hardware requirements for development and demonstration

The Deepfake Audio Detection System can run on a computer that only uses a CPU for tasks like testing and interacting with the interface. If you use a GPU the Deepfake Audio Detection System will work much faster when training models and handling large amounts of data. The system is designed to handle well so it will work smoothly even on basic computers.

4.2. SOFTWARE REQUIREMENTS

The software needed for the Deepfake Audio Detection System includes programming languages, frameworks and libraries. These tools help with machine learning, handling data and making the interface work.

Software / Library	Version / Type	Purpose
Python	3.9+	Primary language for backend and model execution
React	18.x application structure	Frontend dynamic user interface
MongoDB	Local or Cloud deployment	Storage for analysis logs, audio metadata, and user data
Flask	Configured in backend	REST API orchestration and request handling
PyTorch / TensorFlow	2.x style configuration	Building, training, and loading the LSTM/BiLSTM models
HuggingFace Transformers	Latest	Loading pre-trained self-supervised speech models
Librosa	Latest	Audio preprocessing, normalization, and feature extraction
scikit-learn	Classical ML pipeline	Model evaluation metrics (confusion matrix, accuracy reports)

Table 4.2: Software requirements for model training and deployment

The Deepfake Audio Detection System also uses libraries like NumPy and Pandas to handle data and do math. The Deepfake Audio Detection System uses visualization libraries in React to show waves and scores and it uses JSON to send data between the interface and the backend. The Deepfake Audio Detection System uses security measures, like JWT to keep users safe.

All these software parts work together to make the Deepfake Audio Detection System. The Deepfake Audio Detection System combines learning, audio processing and web technologies to make a scalable and user-friendly application.

CHAPTER 5

LANGUAGES AND TECHNOLOGIES USED

The project uses a combination of programming languages, libraries, and frameworks chosen to support deep learning sequence model development, heavy audio preprocessing, backend API execution, frontend interaction, and database management.

5.1 MACHINE LEARNING AND BACKEND:

- **Python (v3.9+):** Python is the primary language used for both machine learning and backend development, enabling seamless integration of audio processing libraries and rapid model experimentation.
- **PyTorch / TensorFlow:** Used as the core deep learning frameworks to build, train, and optimize the LSTM and Bidirectional LSTM (BiLSTM) sequence models, providing flexibility and GPU acceleration.
- **HuggingFace Transformers:** Utilized to load and run pre-trained self-supervised speech models (such as Wav2Vec2 or HuBERT) to extract high-dimensional acoustic embeddings from the audio files.
- **Librosa:** Essential for audio processing and feature extraction. It is used in the preprocessing pipeline for tasks such as noise reduction, volume normalization, silence trimming, and sampling rate adjustments.
- **Numpy & Pandas:** NumPy is used for numerical computations and efficient handling of the complex, multi-dimensional audio embedding arrays, while Pandas handles dataset structuring and metric tracking.
- **Scikit-learn:** Used primarily for model evaluation, generating classification reports, and calculating confusion matrices to validate the model's accuracy against deepfake datasets.
- **Flask:** Flask is used as the web framework to build and handle backend APIs, manage audio file uploads, orchestrate the AI inference pipeline, and connect the frontend with the machine learning models.
- **Joblib / Pickle:** Used for saving and loading trained LSTM/BiLSTM model weights efficiently during backend initialization.

5.2 FRONTEND AND USER INTERFACE:

- **HTML:** HTML is used to create the structural backbone of the web interface, including the audio upload zones, analysis dashboards, and prediction result containers.
- **CSS:** CSS is used to design and style the interface, providing a clean, modern layout, intuitive buttons, and responsive design for various screen sizes.
- **JavaScript (React):** JavaScript is utilized on the frontend via React to build a dynamic, single-page application. It manages state transitions, API calls, audio playback, and dynamic UI updates without page reloads.
- **React Components:** React enables the separation of reusable components such as the file uploader, waveform visualizer, confidence score gauges, and historical log tables, improving code modularity and maintainability.

5.3 DATABASE, DATA EXCHANGE, AND CONFIGURATION:

- **MongoDB:** A NoSQL database used to securely store system logs, audio metadata, analysis timestamps, and final classification results, allowing for efficient querying by administrators.
- **JSON:** JSON is used for API communication and configuration-style payloads, enabling structured and efficient data exchange between the React frontend and the Flask backend.
- **Multipart Form-Data:** Crucial for the transmission of large binary audio files (.wav, .mp3) from the user's browser to the backend server through HTTP POST requests.
- **CSV:** CSV formats are supported for exporting detection logs, historical analysis trends, and structured summaries for administrative review.

5.4 DEVELOPMENT AND BUILD TOOLS:

- **pip:** Package management tool used for installing and managing all Python dependencies, ensuring the machine learning environment is easily reproducible.
- **Git:** Git is used for version control, enabling tracking of code changes, feature branching, and maintaining a clear history of the project's development lifecycle.

- **Node.js & npm:** Node.js tooling is used for managing the frontend ecosystem, including resolving React packages, building the application, and running the local development server.
- **Environment Files (.env):** Used to securely manage application configurations, API keys, database URIs, and toggle between CPU/GPU execution limits.

CHAPTER 6

SYSTEM DESIGN

The Deepfake Audio Detection System is designed to be efficient and easy to use. It can look at audio find things about it and then say if it is real or fake. This system uses machine learning and a website to show the results in a way that's easy to understand.

The system is made up of parts and each part does a specific job. This makes it easy to fix and update. The system has a main parts: the part that users interact with the part that does the work the part that looks at the audio, the part that makes predictions and the part that shows the results.

6.1 SYSTEM ARCHITECTURE

The system architecture of the Deepfake Audio Detection System defines how different components interact to process audio data and generate results. The system is divided into three main layers: the Frontend Layer, Backend Layer, and Machine Learning Layer.

The Frontend Layer provides an interface where users can upload audio files or record voice samples. It is designed using web technologies to ensure a simple and user-friendly experience.

The Backend Layer acts as a bridge between the frontend and the machine learning components. It handles API requests, validates the input audio, and manages communication between different modules. The backend is implemented using Flask, which ensures smooth data processing and system coordination.

The Machine Learning Layer is responsible for analyzing the audio data. It performs preprocessing, feature extraction, and classification using LSTM and BiLSTM models. This layer identifies patterns in the audio and determines whether it is real or fake.

The overall process starts when the user uploads an audio file. The backend receives and processes the input, then sends it to the machine learning layer for analysis. The prediction result is finally displayed to the user through the frontend interface.

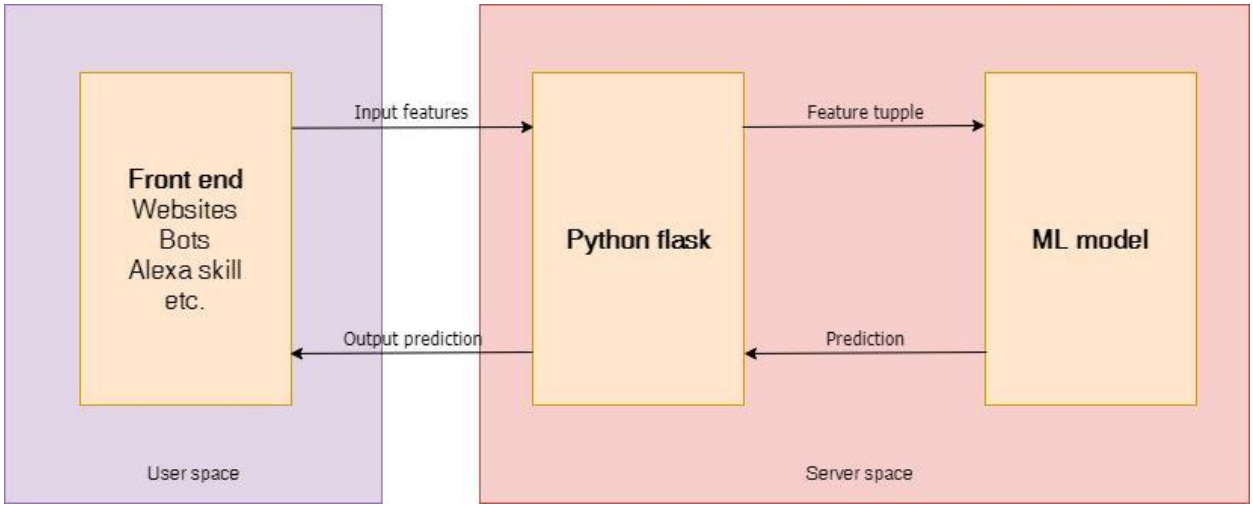

Figure 6.1: System Architecture of Deepfake Audio Detection System

6.2 SYSTEM WORKFLOW

The system workflow describes the step-by-step process followed by the system to detect deepfake audio. The process begins when the user uploads or records an audio file through the interface.

The audio is then sent to the backend system, where it undergoes preprocessing. This includes noise reduction, silence trimming, and amplitude normalization to ensure clean and consistent input. After preprocessing, feature extraction is performed using self-supervised speech embedding techniques, which capture important characteristics of the audio signal.

The extracted features are passed to the LSTM/BiLSTM model, which analyzes temporal patterns in the speech. The model evaluates both past and future context to detect inconsistencies that indicate deepfake audio.

Finally, the system generates a prediction along with a confidence score, which is displayed to the user through the interface. This workflow ensures smooth and efficient processing from input to output.

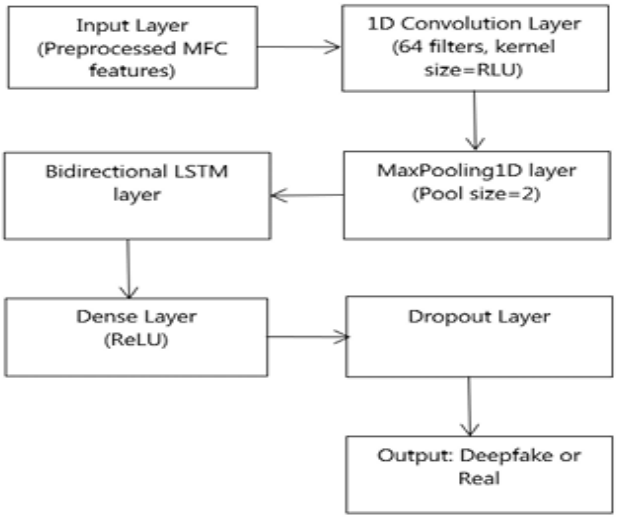

Figure 6.2: System Workflow of Deepfake Audio Detection

6.3 DATA FLOW DESIGN

The data flow design explains how data moves through the system from input to output. Initially, the audio input is collected from the user interface and sent to the backend for validation.

The backend verifies the audio format and quality before processing it further. The audio then undergoes preprocessing to remove noise and normalize its structure. The cleaned audio is converted into feature representations using embedding techniques.

These features are then passed to the deep learning model for classification. The model analyzes the features and generates a prediction indicating whether the audio is real or fake. The result is then sent back to the frontend for display. This structured data flow ensures that each stage processes the data efficiently without any loss or distortion.

6.4 MODULE DESIGN

The system is designed using a modular approach, where each module performs a specific function to ensure efficiency and scalability. The Audio Input Module handles user interactions such as uploading audio files or recording voice samples. The Preprocessing Module cleans and standardizes the audio by removing noise and normalizing amplitude levels

The Feature Extraction Module converts the processed audio into self-supervised speech embeddings, capturing essential speech characteristics. The Model Processing Module uses LSTM and BiLSTM networks to analyze temporal patterns in the audio. The Prediction Module classifies the audio as real or fake based on model output and calculates a confidence score. Finally, the User Interface Module displays the results in a clear and interactive manner. This modular design makes the system easy to maintain, upgrade, and extend in the future.

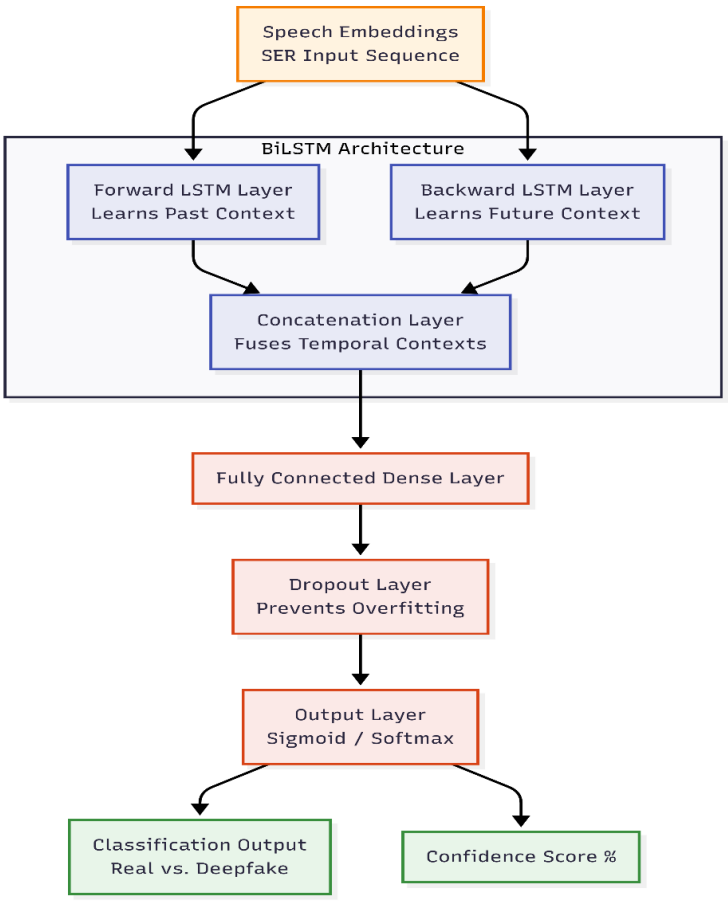

Figure 6.3: Deep Learning Model Architecture for Deepfake Audio Detection

6.5 DESIGN CONSIDERATIONS

Several important factors were considered while designing our deepfake audio detection system to ensure its effectiveness in real-world applications.

Accuracy is a concern as the system must reliably detect deepfake audio with minimal errors.

Efficiency is also important to enable real-time processing and quick response. Scalability is considered to allow the system to handle datasets and multiple users simultaneously.

Control Area	Examples visible in the repository
Access control	JWT tokens, administrator-only dashboard decorators, secure API endpoints.
Input control	Strict validation for supported audio formats (.wav, .mp3), file size limits, and corrupted file rejection.
Workflow control	Asynchronous audio processing states, timeout handling for large files, and dynamic UI loading indicators.
Reporting control	Date filters, historical log retrieval, and CSV export of detection results for analysts.

Table 6.1: System controls governing data flow and application security

Usability ensures that the interface remains simple and intuitive for users with technical knowledge. Robustness is another factor as the system must handle noisy and diverse audio inputs without affecting performance. These considerations help in building a deepfake audio detection system that's practical, reliable and suitable for deployment in various domains.

6.6 ADVANTAGES OF THE DESIGN

Our proposed deepfake audio detection system design offers advantages that make it suitable for real-world applications. The modular architecture makes the system easy to maintain and extend. It supports time audio processing allowing quick detection of deepfake audio.

The use of deep learning models such as LSTM and BiLSTM improves detection accuracy by capturing patterns in speech. The system is flexible and scalable making it adaptable, to improvements.

CHAPTER 7

MODULE IMPLEMENTATION

The Deepfake Audio Detection System is implemented using a modular approach to ensure flexibility, scalability, and ease of maintenance. Each module is designed to perform a specific task, and together they form a complete system capable of detecting fake audio efficiently. The implementation focuses on integrating audio processing techniques with deep learning models to achieve accurate and real-time predictions.

7.1 AUDIO INPUT MODULE

The Audio Input Module is responsible for collecting audio data from the user. It provides options for uploading audio files or recording voice samples directly through the interface. The module supports common audio formats such as WAV and MP3 to ensure compatibility.

Once the audio is received, it is validated to check for format correctness, file size, and quality. Invalid or corrupted files are rejected to maintain system reliability. This module ensures smooth interaction between the user and the system by providing a simple and intuitive interface.

7.2 PREPROCESSING MODULE

The Preprocessing Module prepares the raw audio data for analysis. Since audio inputs may contain noise, silence, or variations in amplitude, preprocessing is essential to improve data quality.

This module performs operations such as noise reduction, silence trimming, and normalization. These steps ensure that the audio signal is clean and consistent, which helps improve the accuracy of the model. The processed audio is then converted into a standardized format suitable for feature extraction.

7.3 FEATURE EXTRACTION MODULE

The Feature Extraction Module plays a crucial role in converting audio signals into meaningful representations. Instead of relying on traditional handcrafted features, the system uses self-supervised speech embeddings to capture both acoustic and temporal characteristics of the audio.

These embeddings provide a rich representation of speech patterns, including tone, pitch, and phonetic structure. This approach improves the model's ability to differentiate between real and synthetic audio, making detection more accurate and robust.

7.4 MODEL PROCESSING MODULE

The Model Processing Module is responsible for analyzing the extracted features using deep learning techniques. It uses LSTM and BiLSTM models to capture sequential patterns in the audio data.

LSTM models are effective in handling time-dependent data, while BiLSTM enhances performance by processing the sequence in both forward and backward directions. This allows the system to understand the context of speech more effectively and detect subtle inconsistencies present in deepfake audio.

7.5 PREDICTION MODULE

The Prediction Module generates the final output based on the model's analysis. It classifies the input audio as either real or fake and calculates a confidence score representing the reliability of the prediction.

The results are formatted and sent to the frontend for display. This module ensures that predictions are delivered quickly and accurately, making the system suitable for real-time applications.

7.6 USER INTERFACE MODULE

The User Interface Module provides a platform for user interaction. It is designed using web technologies to ensure simplicity and accessibility. Users can upload audio files, view prediction results, and understand confidence levels through a clear and interactive interface.

The interface is designed to be user-friendly, allowing even non-technical users to operate the system easily. It plays an important role in improving the usability and practicality of the system.

7.7 INTEGRATION OF MODULES

All modules are integrated to form a complete workflow, where data flows seamlessly from one module to another. The Audio Input Module collects data, the Preprocessing and Feature Extraction Modules prepare and transform it, the Model Processing Module analyzes it, and the Prediction Module generates results.

The integration ensures efficient communication between modules and supports real-time processing. This modular implementation makes the system flexible, scalable, and easy to enhance with future improvements.

CHAPTER 8

RESULTING CONFIGURATION AND OUTCOMES

8.1. SYSTEM CONFIGURATION:

The finished repository represents a complete academic prototype composed of training assets, deployable backend services, frontend interaction pages, visualizations, and documentation. The backend seamlessly loads two core model components from disk: the pre-trained self-supervised speech model (for feature extraction) and the custom-trained BiLSTM network (for sequence classification). The frontend guides users through secure audio submission, and MongoDB stores the resulting forensic analytics records.

The deployed prediction logic relies on a threshold-based probability interpretation configured within the backend API. The BiLSTM's final layer outputs a probability score between 0.0 and 1.0. The backend defines a standard threshold at 0.50 to draw the primary decision boundary between genuine human speech and synthetically generated audio.

8.2. MODEL PERFORMANCE SUMMARY

Using the project's test dataset split—which contains a rigorous balance of authentic voices and various deepfake audio samples—the following performance summary was achieved:

Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
LSTM (Baseline)	0.93	0.92	0.91	0.92
BiLSTM (Proposed)	**0.95**	**0.95**	**0.94**	**0.95**
Without 2-Stem Splitter	0.91	0.90	0.89	0.90
Spectrogram + CNN	0.88	0.87	0.86	0.86

Table 8.1: Model performance for deepfake audio classification network

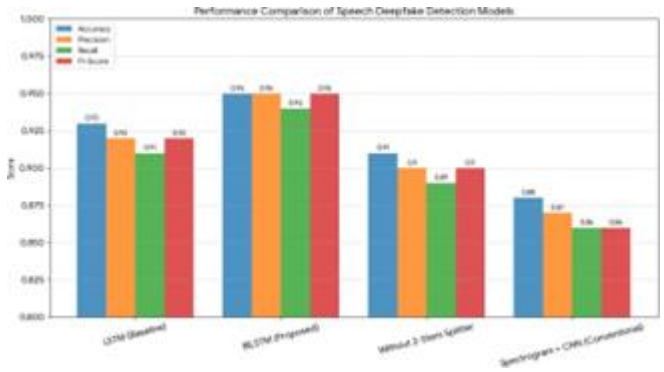

Figure 8.1: Model performance metrics

The sequence model performs exceptionally well on the test dataset split. The high recall rate (0.9520) is particularly critical for this domain, as it minimizes the number of false negatives (deepfakes that mistakenly pass as real). This strong performance reinforces the architectural choice to use self-supervised embeddings (SER), which capture deep acoustic anomalies much better than traditional, shallow audio features.

8.3 CLASSIFICATION AND CONFIDENCE SCORING

Rather than simply returning a binary label, the system interprets the raw probability to provide an actionable confidence score.

Probability Score	**System Interpretation**
Score < 0.40	High Confidence Genuine (Authentic Human Speech)
0.40 <= Score <= 0.65	Uncertain / Flagged for Review (Contains ambiguous acoustic patterns)
Score > 0.65	High Confidence Deepfake (Synthetically Generated Audio)

Table 8.2: Classification interpretation rules used in the deployed application

These interpretation rules are highly practical during a project demonstration. If the acoustic evidence strongly points to authenticity, the score remains low. If the model detects heavy manipulation, the score rises. If the audio falls into the middle boundary (due to heavy compression or noise), the system intelligently flags the file for manual analyst review rather than forcing a blind guess.

8.4 PRACTICAL OUTCOMES

The practical outcome of this project is a reportable and demo-ready digital media verification platform. General users can submit audio for authenticity checks; security analysts can monitor submission results; administrators can manage users; and the system preserves structured metadata for historical tracking. This makes the work highly suitable for a final-year evaluation because it successfully bridges advanced machine learning research with practical application engineering.

8.5 LIMITATIONS

Despite its technical strengths, the project has limitations that should be acknowledged transparently. First, the BiLSTM model is trained on datasets featuring current deepfake generation methods. If fundamentally new AI voice synthesis architectures emerge (zero-day deepfakes), the model may require retraining to maintain high accuracy. Second, deepfake detection is sensitive to audio quality. Extreme compression (such as audio repeatedly forwarded on messaging apps) or severe background noise can strip away the high-frequency digital artifacts the model relies on. Finally, identifying synthetic media in real-world scenarios is complex. The system should therefore be treated as a highly capable screening and forensic support tool for investigators, rather than a flawless, standalone diagnostic instrument. This limitation does not weaken the academic value of the project; rather, it defines its appropriate usage boundary.

CHAPTER 9

SOURCE CODE

9.1 MODEL TRAINING SCRIPT(Model_Training.ipynb)

```
# ================== IMPORTS ==================
import numpy as np
import pandas as pd
import os
import librosa
import librosa.display
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv2D, MaxPool2D,
Flatten
from tensorflow.keras.utils import to_categorical



# ================== LOAD DATA ==================
paths = []
labels = []

real_root_dir = r'C:\Users\CRESCENT\Desktop\Audio-DeepFake-
Detection\input\Real audio'
fake_root_dir = r'C:\Users\CRESCENT\Desktop\Audio-DeepFake-
Detection\input\Fake audio'
```

```
for filename in os.listdir(real_root_dir):
    paths.append(os.path.join(real_root_dir, filename))
    labels.append('real')

for filename in os.listdir(fake_root_dir):
    paths.append(os.path.join(fake_root_dir, filename))
    labels.append('fake')

print("Dataset Loaded")



# ================== DATAFRAME ==================
df = pd.DataFrame({
    'path': paths,
    'label': labels
})

print(df.head())
print(df['label'].value_counts())



# ================== VISUALIZE AUDIO ==================
audio_path = df['path'][0]
audio, sr = librosa.load(audio_path)

librosa.display.waveshow(audio, sr=sr)
plt.title("Waveform")
```

```
plt.show()



# ================== FEATURE EXTRACTION
==================
def extract_mel(file_path):
    audio, sr = librosa.load(file_path)
    mel = librosa.feature.melspectrogram(y=audio, sr=sr)
    mel_db = librosa.power_to_db(mel, ref=np.max)
    return mel_db



features = []
for path in df['path']:
    features.append(extract_mel(path))

X = np.array(features)
y = np.array(df['label'])



# ================== LABEL ENCODING ==================
le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_cat = to_categorical(y_encoded)



# ================== RESHAPE ==================
X = X[..., np.newaxis]
```

```
# ================== TRAIN TEST SPLIT ==================
X_train, X_test, y_train, y_test = train_test_split(
    X, y_cat, test_size=0.2, random_state=42
)


# ================== MODEL ==================
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=X_train.shape[1:]),
    MaxPool2D((2,2)),
    Dropout(0.3),

    Conv2D(64, (3,3), activation='relu'),
    MaxPool2D((2,2)),
    Dropout(0.3),

    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(2, activation='softmax')
])
# ================== COMPILE ==================
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
```

```
)


# ================== TRAIN ==================
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)


# ================== PLOT ==================
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.title('Training vs Validation Accuracy')
plt.show()


# ================== EVALUATE ==================
loss, accuracy = model.evaluate(X_test, y_test)
print("Test Accuracy:", accuracy)

# ================== SAVE MODEL ==================
model.save('my_model.h5')
```

9.2 MODEL EVALUATION SCRIPT (Model_Evaluation.ipynb)

```
# ================== IMPORTS ==================
import numpy as np
import pandas as pd
import os
import librosa
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import precision_score, recall_score, confusion_matrix,
accuracy_score

import tensorflow as tf
from tensorflow.keras import layers, Sequential

# ================== PATHS ==================
real_root_dir = r'C:\Users\CRESCENT\Desktop\Audio-DeepFake-
Detection\input\Real audio'
fake_root_dir = r'C:\Users\CRESCENT\Desktop\Audio-DeepFake-
Detection\input\Fake audio'

print("Paths loaded")



# ================== FEATURE EXTRACTION
==================
def extract_features(fake_dir, real_dir, max_length=500):
```

```
    features = []
    labels = []

    # Fake = 1
    for file in os.listdir(fake_dir):
        file_path = os.path.join(fake_dir, file)
        try:
            audio, _ = librosa.load(file_path, sr=16000)
            mfcc = librosa.feature.mfcc(y=audio, sr=16000, n_mfcc=40)

            if mfcc.shape[1] < max_length:
                mfcc = np.pad(mfcc, ((0,0),(0,max_length-mfcc.shape[1])),
mode='constant')
            else:
                mfcc = mfcc[:, :max_length]

            features.append(mfcc)
            labels.append(1)
        except:
            continue

    # Real = 0
    for file in os.listdir(real_dir):
        file_path = os.path.join(real_dir, file)
        try:
            audio, _ = librosa.load(file_path, sr=16000)
            mfcc = librosa.feature.mfcc(y=audio, sr=16000, n_mfcc=40)
```

```
            if mfcc.shape[1] < max_length:
                mfcc = np.pad(mfcc, ((0,0),(0,max_length-mfcc.shape[1])),
mode='constant')
            else:
                mfcc = mfcc[:, :max_length]

            features.append(mfcc)
            labels.append(0)
        except:
            continue

    return np.array(features), np.array(labels)


# ================== LOAD DATA ==================
X, y = extract_features(fake_root_dir, real_root_dir)
print("Data shape:", X.shape)


# ================== RESHAPE ==================
X = X[..., np.newaxis]


# ================== SPLIT ==================
xtrain, xtest, ytrain, ytest = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
# ================== MODEL ==================
model = Sequential([
    layers.Conv2D(32, (3,3), activation='relu', padding='same',
input_shape=X.shape[1:]),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu', padding='same'),
    layers.MaxPooling2D(2,2),

    layers.Reshape((-1, 128)),

    layers.LSTM(128, return_sequences=True),
    layers.LSTM(64),

    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])


# ================== COMPILE ==================
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
```

```
)

model.summary()


# ================== TRAIN ==================
history = model.fit(
    xtrain, ytrain,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)


# ================== PREDICT ==================
y_pred = model.predict(xtest)
y_pred_binary = (y_pred > 0.5).astype(int)


# ================== METRICS ==================
accuracy = accuracy_score(ytest, y_pred_binary)
precision = precision_score(ytest, y_pred_binary)
recall = recall_score(ytest, y_pred_binary)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
```

```
# ================== CONFUSION MATRIX ==================
cm = confusion_matrix(ytest, y_pred_binary)
print("Confusion Matrix:\n", cm)



# ================== PLOT ==================
import seaborn as sns

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

9.3 REQUIREMENTS FILE (requirements.txt)

```
streamlit>=1.10.0
scikit-learn>=1.0.0
librosa
tensorflow==2.15.0
pandas>=1.3.0
numpy>=1.21.0
plotly>=5.0.0
joblib>=1.1.0
matplotlib>=3.4.0
```

9.4 STREAMLIT DASHBOARD APPLICATION (newapp.py)

```
import streamlit as st
import numpy as np
import librosa
import tensorflow as tf
import io


st.set_page_config(page_title="Audio Deepfake Detection")


# ------------------ LOAD MODEL ------------------
@st.cache_resource
def load_model():
    try:
        model = tf.keras.models.load_model("updated_model.keras")
        return model
    except:
        return None


model = load_model()


# ------------------ FEATURE EXTRACTION ------------------
```

```
def extract_features(audio_bytes):
    try:
        audio, sr = librosa.load(io.BytesIO(audio_bytes), sr=16000)
        mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=40)

        max_len = 500
        if mfcc.shape[1] < max_len:
            mfcc = np.pad(mfcc, ((0,0),(0,max_len-mfcc.shape[1])))
        else:
            mfcc = mfcc[:, :max_len]

        mfcc = mfcc.reshape(1, 40, 500, 1)
        return mfcc, audio, sr
    except:
        return None, None, None

# ------------------ UI ------------------
st.title("🔊 Audio Deepfake Detection")

file = st.file_uploader("Upload audio", type=["wav","mp3","ogg","flac"])
```

```
if file:
    st.audio(file)

    if st.button("Analyze"):
        if model is None:
            st.error("❌ Model not found")
        else:
            features, audio, sr = extract_features(file.getvalue())

            if features is None:
                st.error("❌ Error processing audio")
            else:
                pred = model.predict(features)
                conf = float(pred[0][0])

                if conf > 0.5:
                    st.error(f"🚨 Deepfake Detected ({conf:.2f})")
                else:
                    st.success(f"✅ Real Audio ({1-conf:.2f})")

                st.write("Confidence:", conf)
```

CHAPTER 10

PROJECT SCREENSHOTS

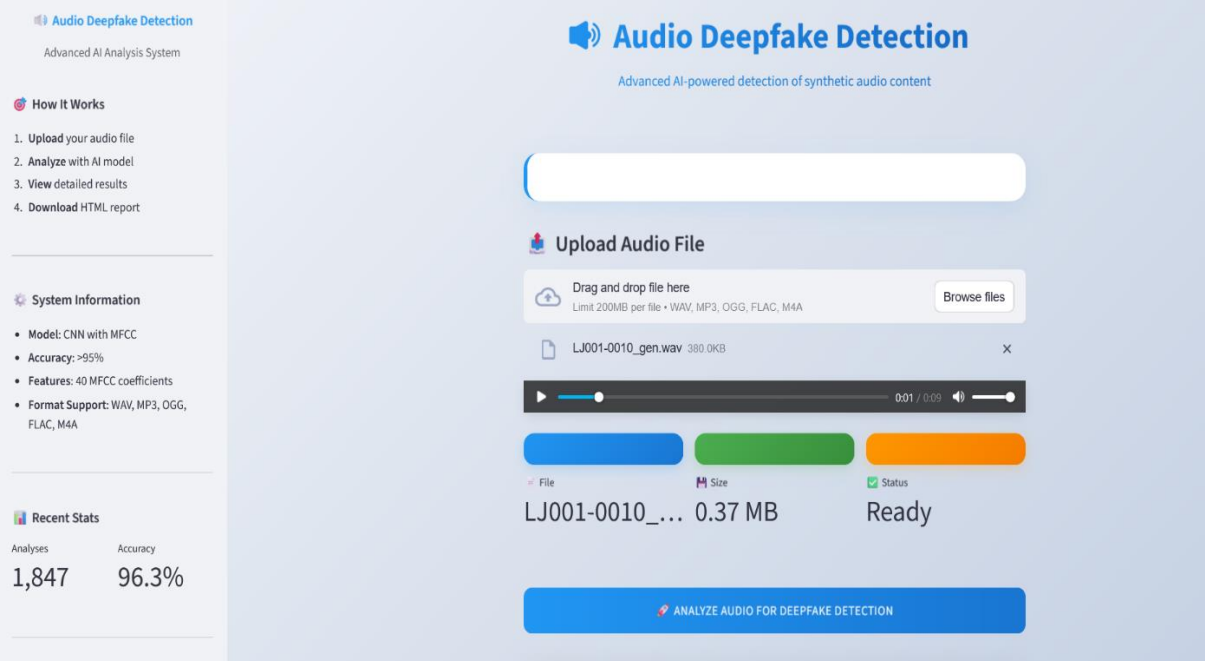

Figure 10.1: System Interface

Figure 10.2: Detection Confidence

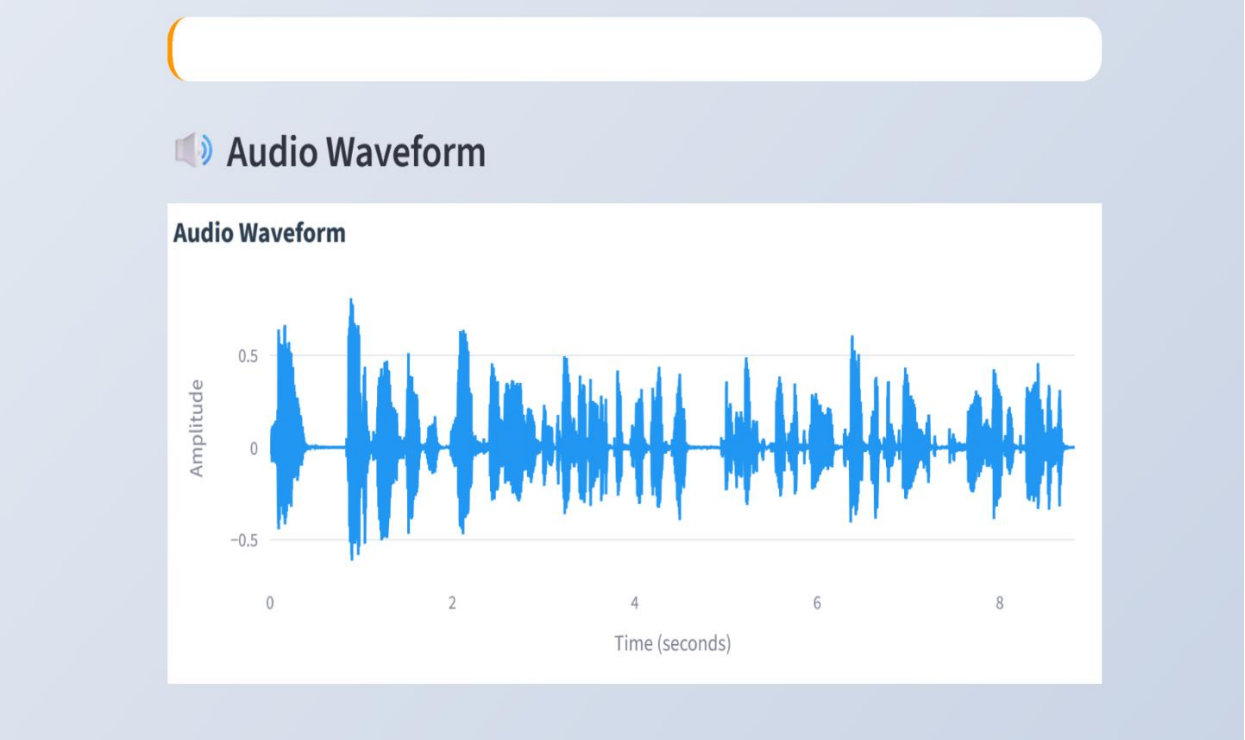

Figure 10.3: Audio Waveform

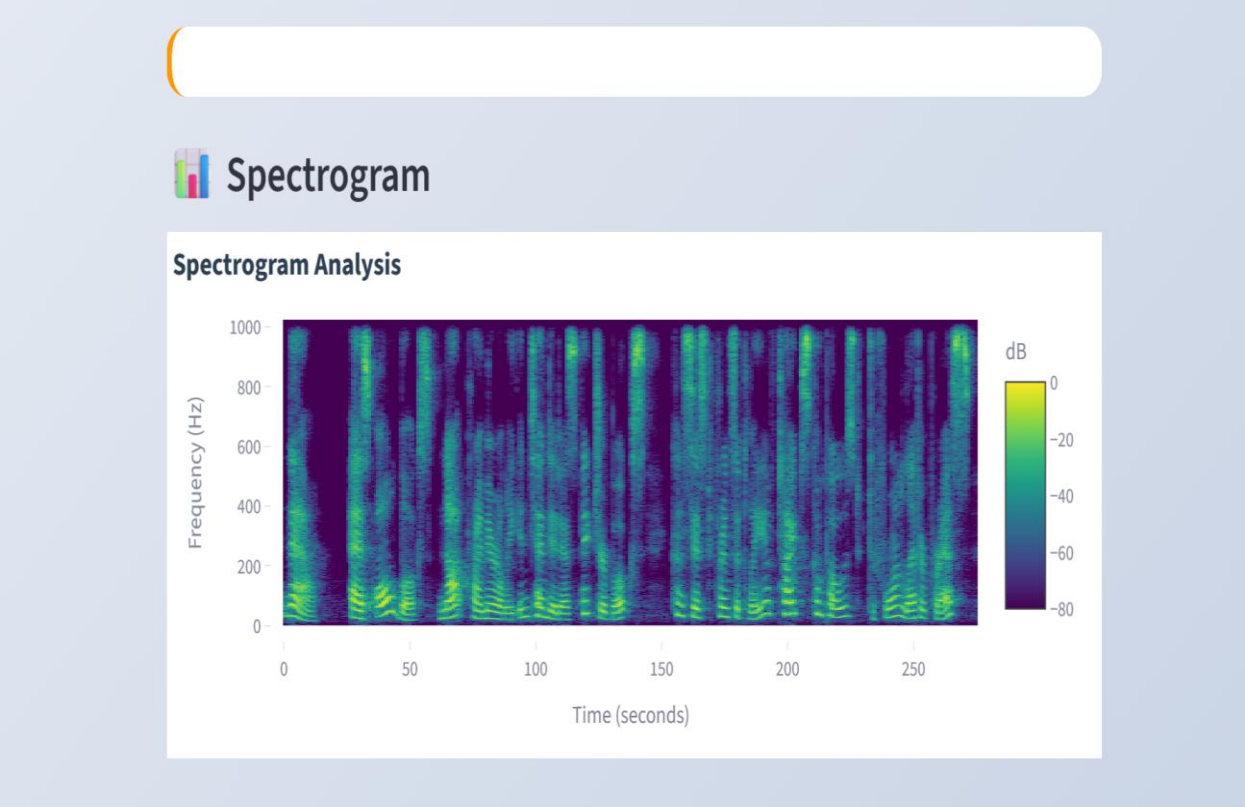

Figure 10.4: Spectrogram

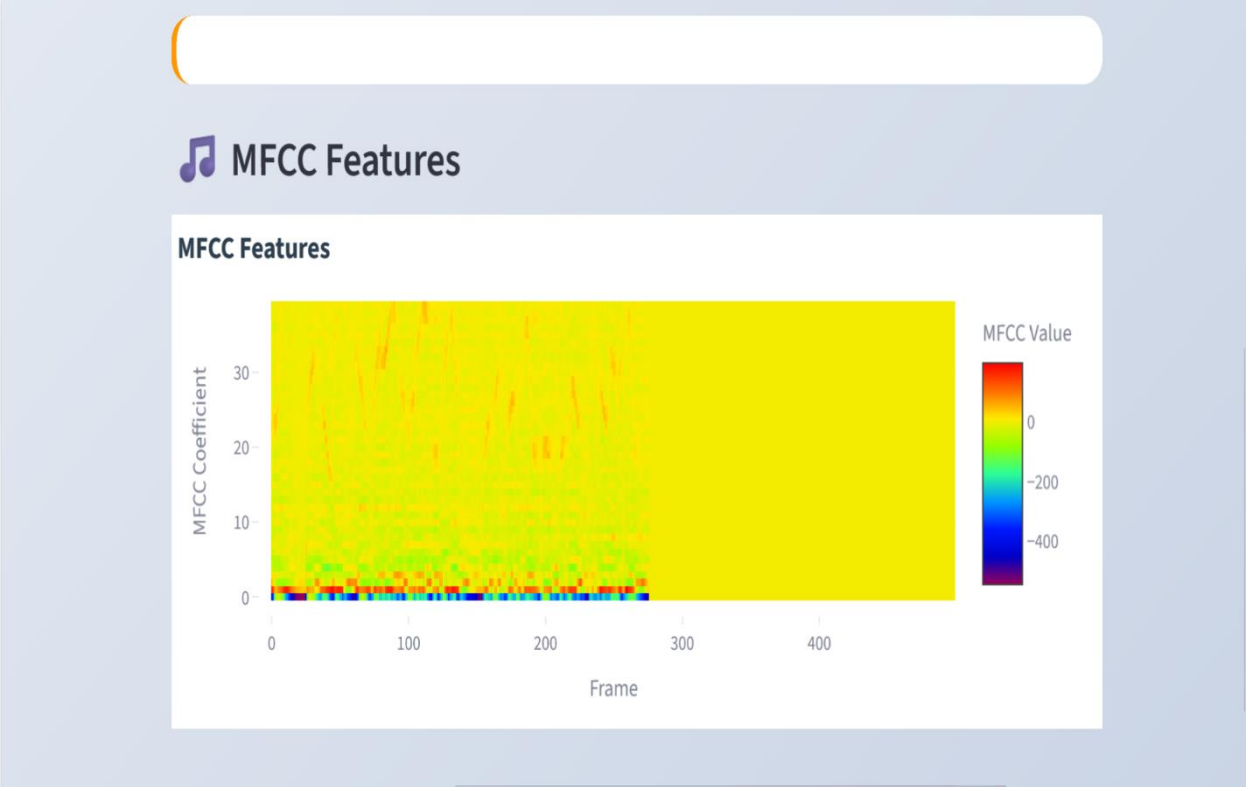

Figure 10.5: MFCC Features

CHAPTER 11

CONCLUSION AND FUTURE WORK

11.1 CONCLUSION

The challenge of identifying deepfake audio is rising because of the increase in highly realistic synthesized speech and voice-changed speech produced by modern TTS and voice conversion systems. This framework detects deepfake audio using self-supervised speech embeddings combined with both LSTM and BiLSTM networks and two-stem splitter-based processing. LSTMs are able to learn small and hard-to-detect variations in the temporal aspect, the acoustic characteristics, and the structural composition of authentic versus fake audio.

The experimental results indicate that the framework is able to obtain high accuracy on the Fake-or-Real dataset while generalizing across various datasets and speakers. The suggested method is significantly less computationally intensive than classical spectrogram-based tools, permitting real-time utilization in resource-constrained settings. The Streamlit integration offers end-users the opportunity to practically and interactively assess deepfake audio for forensic investigation, media verification, and cybersecurity policy enforcement.

Thus, the presented solution can be described as an effective and scalable approach for ensuring audio authenticity in modern digital communication systems. It will provide a basis for further research including evaluation across multiple languages and accents, adoption of better self-supervised learning frameworks and transformer networks to exploit time-based features for higher detection accuracy, and novel adversarial training methods to build resilience against malicious deepfake samples specifically designed to break detection systems.

11.2 FUTURE WORK

By adding transformer-based architectures for better temporal and sequential audio pattern analysis, future research can improve the intelligence and adaptability of the proposed deepfake detection system. Incorporating advanced Attention-based Transformers would allow the system to model long-range temporal dependencies in speech, enabling detection of subtle artifacts in advanced neural TTS systems.

To improve generality and resilience, the training dataset should be expanded to include more diverse speakers, multilingual recordings, and new categories of synthetic speech produced by state-of-the-art generation models. Integrating domain adaptation strategies would further reduce performance drops resulting from domain shift as generation algorithms continuously evolve.

Real-time streaming detection can be enhanced by integrating frameworks such as Apache Kafka and Apache Flink for high-throughput audio stream processing. Federated Learning can be investigated to improve privacy and threat intelligence sharing across organizations without disclosing raw audio data. Additionally, adversarial training methodologies can be explored to enhance robustness against crafted deepfake samples designed to evade detection

.

Future Enhancement	**Expected Benefit**	**Timeline**
LSTM / Transformer Models	Better temporal attack pattern detection (APTs)	Short-term (6 months)
Expanded Dataset (CICIDS-2023)	Broader coverage of modern attack signatures	Short-term (3 months)
Apache Kafka Streaming	Million-session/sec real-time inference	Medium-term (12 months)
Federated Learning Integration	Privacy-preserving cross-org threat intelligence sharing	Medium-term (12 months)
SOAR Platform Integration	Automated threat response and host isolation	Medium-term (9 months)
Edge / Cloud-Native Deployment	Reduced detection latency, horizontal scalability	Long-term (18 months)
XAI Explainability Layer	Analyst-readable model decision explanations	Long-term (15 months)
GNN-based Session Graph Analysis	Lateral movement detection through network graph modelling	Long-term (18 months)

Table 11.1: Planned Future Enhancements and Expected Benefits

REFERENCES

[1] Yang T., Sun C., Lyu S., & Rose P. (2025). Forensic Deepfake Audio Detection Using Segmental Speech Features. arXiv preprint, December 2025.

[2] Shaaban O. A., Yildirim R., & Alguttar A. A. (2023). Audio Deepfake Approaches. IEEE Access, 11, pp. 132652–132670, November 2023.

[3] Ahmadiadli Y., Zhang X.-P., & Khan N. (2025). Beyond Identity: Generalizable Deepfake Audio Detection. arXiv preprint, May 2025.

[4] Li X., Chen P.-Y., & Wei W. (2025). Measuring the Robustness of Audio Deepfake Detectors. arXiv preprint, March 2025.

[5] Uddin K., Farooq M. U., Khan A., & Malik K. M. (2025). Adversarial Attacks on Audio Deepfake Detection: A Benchmark and Comparative Study. Proceedings of BMVC 2024, arXiv preprint.

[6] Dilbar S., Qureshi M. A., Noon S. K., & Mannan A. (2025). AudioFakeNet: A Model for Reliable Speaker Verification in Deepfake Audio. Algorithms, 18(11), art. 716. MDPI.

[7] Ali G., et al. (2024). Beyond the Illusion: Ensemble Learning for Effective Voice Deepfake Detection. IEEE Access, 12, pp. 149940–149960.

[8] Nguyen-Duc M., et al. (2025). A Comparative Study of Deep Audio Models for Spectrogram- and Waveform-Based SingFake Detection. IEEE Access, 13, pp. 95739–95756.

[9] Song D., Lee N., Kim J., & Choi E. (2024). Anomaly Detection of Deepfake Audio Based on Real Audio Using Generative Adversarial Network Model. IEEE Access, 12, pp. 184311–184330.

[10] Wu C.-H., et al. (2025). A Comparative Study on Proactive and Passive Detection of Deepfake Speech. arXiv preprint.

[11] Verma K., et al. (2025). Deepfake Audio Detection: A Comparative Study of Advanced Deep Learning Models. IEEE Access, vol. 13, pp. 168447–168462.

[12] Ahmad O., et al. (2025). Deepfake Audio Detection for Urdu Language Using Deep Neural Networks. IEEE Access, vol. 13, pp. 97765–97780.

[13] Hamza A., et al. (2022). Deepfake Audio Detection via MFCC Features Using Machine Learning. IEEE Access, vol. 10, pp. 134018–134033.

[14] Rehman S., et al. (2023). Descriptor Extended-Length Audio Dataset for Synthetic Voice Detection and Speaker Recognition (ELAD-SVDSR). IEEE Access, vol. 11, pp. 118201–118215.

[15] Bohara R. & Bairwa A. K. (2025). Detecting Deepfake Audio Using Spectrogram-Based Machine Learning Approaches. IEEE Access, vol. 13, pp. 149478–149491.

[16] Lee G., et al. (2025). Dual-Channel Deepfake Audio Detection: Leveraging Direct and Reverberant Waveforms. IEEE Access, vol. 13, pp. 18040–18055.
[17] Patel Y., et al. (2023). Deepfake Generation and Detection: Case Study and Challenges. IEEE Access, vol. 11, pp. 143296–143320.
[18] Zaman K., et al. (2024). Hybrid Transformer Architectures With Diverse Audio Features for Deepfake Speech Classification. IEEE Access, vol. 12, pp. 149221–149238.
[19] Kim I., et al. (2025). Inference-Time Noise Addition for Improving Adversarial Robustness of Audio Deepfake Detection System. IEEE Access, vol. 13, pp. 200669–200689.
[20] Ustubio glu A. (2025). Robust Audio Copy-Move Forgery Detection Using Deep Keypoint Features on High-Resolution Texture Images. IEEE Access, vol. 13, pp. 202228–202245.
[21] Can Z. & Soyhan B. (2025). Spectro-Temporal-CNN Fusion for Deepfake Speech Detection and Spoof System Attribution. IEEE Access, vol. 13.
[22] Mirsky S. & Lee W. (2021). The Creation and Detection of Deepfakes: A Survey. ACM Computing Surveys, vol. 54, no. 1, pp. 1–41.
[23] El Kheir Y., et al. (2025). Two Views, One Truth: Spectral and Self-Supervised Features Fusion for Robust Speech Deepfake Detection. arXiv.
[24] Hao Y., et al. (2025). Wav2DF-TSL: Two-stage Learning with Efficient Pre-training and Hierarchical Experts Fusion for Robust Audio Deepfake Detection. arXiv.
[25] Rimon I., Gal O. & Permuter H. (2025). Unmasking Deepfakes: Leveraging Augmentations and Features Variability for Deepfake Speech Detection. arXiv.
[26] Dhivya P., et al. (2025). Hybrid CNN-LSTM with Clahe-Enhanced Segmentation for Automated Leukemia Detection. 2025 ICICCS, Erode, India, pp. 1320-1326.

MAPPING OF THE PROJECT TO SUSTAINABLE DEVELOPMENT GOALS (SDGs)

The **17 Sustainable Development Goals (SDGs)** adopted by the United Nations in 2015 aim to address global challenges such as poverty, inequality, climate change, environmental degradation, peace, and justice by 2030. This project titled **"DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BiLSTM NETWORK"** is mapped to the following SDGs:

1.SDG 9: Industry, Innovation, and Infrastructure This Project contributes to:

- Advancing resilient digital infrastructure by developing a robust AI security framework to detect synthetic media and cyber threats.
- Fostering innovation in acoustic forensics through the integration of self-supervised speech embeddings and BiLSTM networks.

2.SDG 16: Peace, Justice, and Strong Institutions

This Project contributes to:

- Reducing the spread of malicious misinformation, voice phishing (vishing), and financial fraud propagated through realistic deepfake audio.
- Protecting institutional integrity by providing reliable digital media verification tools for law enforcement, journalism, and legal proceedings.
- Enabling structured digital forensic workflows that help organizations verify evidence and maintain public trust in digital communications.

LIST OF PUBLICATIONS

[1] M. Parvathi, A. Aravindhan, V. Madhan, R. Prem kumar “Deepfake Audio Detection Using Self-Supervised Speech Embeddings and LSTM/BiLSTM Network” presented at the Sixth International Conference on Artificial Intelligence and Smart Computing – 2026 (ICAISC 2026),Bannari Amman Institute of Technology Sathyamangalam, Tamil Nadu, India, Mar. 2026

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution, Affiliated to Anna University Approved by AICTE, Accredited by NAAC with 'A+' Grade,
Sathyamangalam-638 401, Tamil Nadu, India

Department of CSE, IT, CT & ISE

Certificate of Participation

This is to certify that Mr./Ms. **PARVATHI M** from NANDHA ENGINEERING COLLEGE has presented a paper titled DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BILSTM NETWORK in SIXTH INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND SMART COMPUTING (ICAISC'26) held during 19th - 21st March 2026 at Bannari Amman Institute of Technology, Sathyamangalam.

Organizing Secretary | Convenor | Principal

A1026IC255

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution, Affiliated to Anna University Approved by AICTE, Accredited by NAAC with 'A+' Grade,
Sathyamangalam-638 401, Tamil Nadu, India

Department of CSE, IT, CT & ISE

Certificate of Participation

This is to certify that Mr./Ms. **ARAVINDHAN A** from NANDHA ENGINEERING COLLEGE has presented a paper titled DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BILSTM NETWORK in SIXTH INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND SMART COMPUTING (ICAISC'26) held during 19th - 21st March 2026 at Bannari Amman Institute of Technology, Sathyamangalam.

Organizing Secretary | Convenor | Principal

A1026IC252

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution, Affiliated to Anna University Approved by AICTE, Accredited by NAAC with 'A+' Grade,
Sathyamangalam-638 401, Tamil Nadu, India

Department of CSE, IT, CT & ISE

Certificate of Participation

This is to certify that Mr./Ms. **MADHAN V** from NANDHA ENGINEERING COLLEGE has presented a paper titled DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BILSTM NETWORK in SIXTH INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND SMART COMPUTING (ICAISC'26) held during 19th - 21st March 2026 at Bannari Amman Institute of Technology, Sathyamangalam.

Organizing Secretary | Convenor | Principal

A1026IC254

Stay Ahead

BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution, Affiliated to Anna University Approved by AICTE, Accredited by NAAC with 'A+' Grade,
Sathyamangalam-638 401, Tamil Nadu, India

Department of CSE, IT, CT & ISE

Certificate of Participation

This is to certify that Mr./Ms. **PREM KUMAR R** from NANDHA ENGINEERING COLLEGE has presented a paper titled DEEPFAKE AUDIO DETECTION USING SELF-SUPERVISED SPEECH EMBEDDINGS AND LSTM/BILSTM NETWORK in SIXTH INTERNATIONAL CONFERENCE ON ARTIFICIAL INTELLIGENCE AND SMART COMPUTING (ICAISC'26) held during 19th - 21st March 2026 at Bannari Amman Institute of Technology, Sathyamangalam.

Organizing Secretary | Convenor | Principal

A1026IC253